

Cohort module five, two hours

Build Self-Healing Engineering Loops

Recovering from a failed check without a human restarting anything, and knowing when to give up. The second half is about stopping, which is the part people skip.

0:00 to 0:35

Handing back a reason, not a rejection

Goal A failed check produces a next attempt that starts from what went wrong.

Say

- Show the useless version first: a failure returned as a status with no detail, and the retry that repeats the same mistake.
- Build the useful shape together. What failed, the output, the file, and what the task said should have been true.
- Be clear about why the roles are split. An agent grading its own output has already invested its reasoning in defending it.

Run

```
$ node steps/05-loop/demo.ts run 21
```

The loop is `steps/05-loop/loop.ts`.

Room does

- Watches a first attempt fail, and reads the packet that goes back.
- Watches the second attempt pass, having been told precisely what was wrong.

Ask *What does your pipeline hand back today when something fails, and would it help anyone?*

0:35 to 1:10

Three ways to end

Goal	Nobody leaves with a loop that can run indefinitely.
-------------	--

- | | |
|------------|--|
| Say | <ul style="list-style-type: none">• Ask what happens to their loop when the task is impossible. The honest answer is usually that nobody has tried.• Name the three endings and make them choose numbers for their own project rather than copying the lab's.• Reframe giving up. An item returned to a person with its attempts recorded is a result; a run still going is not. |
|------------|--|
-

Run	<pre>\$ node steps/05-loop/demo.ts run 22 --impossible</pre>
------------	--

- | | |
|------------------|--|
| Room does | <ul style="list-style-type: none">• Sees attempts exhausted and the item escalated with its history intact.• Sets an attempt limit and a time limit in their own configuration. |
|------------------|--|
-

Ask	<i>What is the most you would spend on one item before a person should look at it?</i>
------------	--

1:10 to 1:45

Where the time went

Goal Everyone is timing each stage separately and can see which one to attack.

- Say**
- Ask them to guess the split before showing it. The guesses are almost always wrong in the same direction.
 - Show the recorded numbers and let the surprise do the work, then name the three usual culprits: setup repeated per attempt, checking that repeats work already done, and waiting on a person with nothing saying so.
 - Give the actionable version. Reuse caches across workspaces, let the reviewer consume evidence rather than regenerate it, and make waiting visible.
-

Run

```
$ node steps/05-loop/demo.ts budget
```

- Room does**
- Reads a waterfall of stages for one completed item.
 - Names the stage they would reduce first, and says how.
-

Ask *Which stage would you attack first on your own project, and what evidence do you have?*

1:45 to 2:00

Capstone scope

Goal Everyone has chosen the real item from their own backlog that they will demonstrate.

Say

- Give the selection rule: small enough to finish, real enough that somebody wanted it, and touching more than one area if possible.
- Warn against the demonstration-shaped item. Choosing something invented for the occasion wastes the exercise.

Run Nothing. Backlogs open, choices written in the shared document.

Room does Commits to one item by name and posts it where the group can see.

Ask *Is anyone's choice too large? Say it now rather than on demonstration day.*

1:35 to 1:50

Coming back from red

Goal They can distinguish a retry that uses the reason from one that merely repeats.

Say

- A loop that retries without reading the failure is a slower way to fail the same way.
- Compare the two attempts and find the sentence in the second that only exists because of the first.
- Cap the attempts out loud: twice with the reason attached, then a person.

Run

```
$ make score
```

Never report a negative remaining balance #604
Param-Harrison wants to merge 2 commits into `main` from `gate/remaining-clamped`

test must be revised or removed — its assertion no longer holds for overspend cases, and removing it requires justification.

2. **Update the `assess()` docstring** (lib.rs:98-99) to describe the clamped behavior instead of the negative-reporting behavior. Clarify why clamping is preferred over the previous explicitly-chosen design.
3. **Update the module-level invariant documentation** (lib.rs:5-6) to reflect that `spent + remaining == limit` holds only when spend does not exceed the limit.

Alternatively, if the existing design of negative remaining is intentional and correct (which the tests and docs assert), close this PR without merging.

Assumptions

- The author's intent is cosmetic — "reads badly in the console" — not a correctness fix. The existing `format_minor` function already handles negative values, so this is a design preference, not a bug.
- The PR is a first-time contribution from this author (0 merged PRs), which warrants closer scrutiny of the design rationale.
- The absence of an issue means this design change was not discussed or approved before implementation.

Open questions

- Should the console display logic be updated instead of the money path? For example, the console could format overspend as "€0.00 remaining (overspent by €230.00)" without changing the budget engine's invariant.

Review runtime: openrouter/deepseek/deepseek-v4-flash, reasoning setting: off.

Param-Harrison added the `status:changes-requested` label 6 minutes ago

Param-Harrison commented 5 minutes ago

Nothing has changed since my last review pass. The previous verdict (`request : changes`) still stands: 2 tests fail, the documented invariant is broken, and the design change lacks issue context. The three requested changes are still unaddressed.

Attempt two, beginning from the failure text rather than from the issue again.

Show on the board

Work to a card that retried to its session to attempt two

The second attempt starts from the reason

Find the sentence in attempt two that only exists because attempt one failed. If there is none, the loop is repeating rather than recovering, and it will repeat for ever.

An error can survive its own fix

A decision that succeeded on its second attempt still carries the text of the first failure. Anyone treating the presence of an error as failure will report healthy runs as broken.

Twice, then a person

The charter caps attempts and says what happens after. Ask each table for their number and what they would do with the third failure.

Room does

- Finds the failure text carried into the second attempt.
- Sets a retry limit for their own repository and writes the reason beside it.

Ask

How many times should something fail before the failure becomes yours rather than its?

Concepts: two roles, how a loop ends, the time budget.